

LinkVault · Technical Presentation Roadmap

3-Member Screen-by-Screen Walkthrough & Mentor Defense Guide

Stack: Flask 3.1 · Python 3.12 · PostgreSQL · SQLAlchemy 2.0 · Unicorn · Render Cloud
Structure: Member 1 (The Entryway) → Member 2 (The Core Product) → Member 3 (The Engine & Cloud)

Member / Speaker	Presentation Domain	Live Screens & Components Demonstrated	Time
Member 1 The Entryway	Frontend UI/UX, Onboarding & Auth Security	• Landing Page (landing.html) • Register & Login Pages (login.html) • Password hashing (PBKDF2-SHA256) & sessions	2.0 min
Member 2 The Core Product	Dashboard System, Search & Team Hub	• Home Dashboard & Categories (index.html) • Instant Search (inline X clear & Escape reset) • Team Hub Sidebar & Workspaces (workspace.html)	2.5 min
Member 3 The Engine & Cloud	Backend APIs, ORM Schema & Cloud Hosting	• Flask Application Factory (app.py) • Relational Models & Cascades (models.py) • PostgreSQL Pooling & Live Render Deployment	2.5 min

1. MEMBER 1: The Entryway (Landing Page & Authentication)

Role: Frontend UI/UX Architect & Authentication Specialist

Live Screens to Show:

- **Landing Page (/ logged out):** Clean hero section, feature grid, and zero-framework design tokens in templates/landing.html.
- **Register Page (/register):** Input fields, client-side validation, error handling for duplicate emails in templates/register.html.
- **Login Page (/login):** Form submission, flash feedback, and seamless redirect to authenticated dashboard in templates/login.html.

Key Technical Explanations:

- **Design System:** Vanilla CSS variables (--bg-primary, --accent-primary) and Google Fonts (Inter) providing high contrast with zero CSS framework bloat.
- **PBKDF2-SHA256 Password Hashing:** Passwords hashed in models.py via Werkzeug; raw passwords are never stored.
- **Flask-Login Session Model:** Signed HTTP-only session cookies with @login_required route authorization.

■■ MEMBER 1 SPEAKING SCRIPT (Word-for-Word Walkthrough)

"Good morning, Sir/Madam. I will be introducing LinkVault and walking you through our user onboarding and authentication architecture. LinkVault is designed to eliminate bookmark clutter and empower team resource sharing. When visitors first arrive, they see our responsive **Landing Page** (landing.html), built using custom CSS tokens and Inter typography for optimal contrast and zero library overhead. From here, users can create an account or sign in. Behind our **Login and Registration forms** (login.html), security is strictly enforced. We hash all passwords using salted PBKDF2-SHA256 in models.py before they touch the database. Once authenticated, Flask-Login creates a secure session cookie protecting our internal routes. Now that we have successfully logged in, I will hand over to [Member 2] to demonstrate our core dashboard, search engine, and team collaboration system."

2. MEMBER 2: The Core Product (Home Dashboard & Team Workspaces)

Role: Product Experience Architect & Team Collaboration Engineer

Member 2 takes over immediately upon login to demonstrate the main user workspace, real-time search, and multi-tenant team collaboration.

Live Screens & Actions to Demonstrate:

- **Home Page Dashboard (/ logged in):** Link cards, category pills (Work, Dev, Study, Personal), pinned/starred toggles in `templates/index.html`.
- **Add Link Modal:** Click + Add Link, input a title and URL, and observe instant card rendering without page reload.
- **Instant Search & Reset:** Type in search box → show real-time filtering → click the X clear button or press Escape key to reset instantly.
- **Team Workspace Hub (/teams & /teams/<id>):** Click the dedicated **Team Workspace button** in the left sidebar (above Categories), view team cards, copy invite codes, and see shared team links.

Key Technical Explanations:

- **DOM Filtering with `replaceState`:** Search query updates the URL with `window.history.replaceState` so browser refreshes keep state, eliminating full-page reload flickers.
- **Sidebar Navigation UX:** Prominent Team Workspace button placed in the sidebar directly above Categories for 1-click context switching between personal and team links.
- **Cryptographic Invite Tokens:** Teams generate 8-character hex codes (`secrets.token_hex(4).upper()`) allowing instantaneous joining with role verification (Owner, Admin, Member).

■■ MEMBER 2 SPEAKING SCRIPT (Word-for-Word Walkthrough)

"Thank you [Member 1]. I will now demonstrate our **Home Dashboard and Team Collaboration Hub**.

Upon authentication, the user is redirected to the **Home Dashboard** (`templates/index.html`). Here, links are organized visually with favicons, categories, and quick toggles for starring and pinning.

We implemented two major architectural features here:

1. Real-Time Search & History Synchronization: Users can instantly search across titles, URLs, and categories. We added an inline clear button (X) and Escape key listener. Crucially, we use `history.replaceState` so query parameters stay updated in the browser without causing page reloads or broken back-button states.

2. Team Workspaces: In the left sidebar, right above our category list, we built a dedicated **Team Workspace** button. When clicked, it takes us to `templates/teams/workspace.html`. In this workspace, members share resources collaboratively, generate 8-character cryptographic invite codes, and manage permissions with Role-Based Access Control.

Now, [Member 3] will explain the backend routing logic, relational database models, and our live production deployment on Render."

3. MEMBER 3: The Engine & Cloud (Backend Logic, Database & Live Running)

Role: Systems Architect, Database Administrator & Cloud DevOps Engineer

Member 3 shows what happens behind the scenes: request orchestration in Flask, relational database schema in PostgreSQL, and live production deployment on Render.

Exact Files & Infrastructure to Show On-Screen:

- **Relational Schema (models.py):** 4 core models: User, Team, TeamMember, and Link with cascade foreign keys.
- **Database Engine & Pooling (app.py L55–125):** Normalization (postgresql+psycopg://), SSL enforcement (sslmode=require), and pool settings (pool_pre_ping=True, pool_recycle=1800).
- **Cold-Start Recovery Hook (app.py L105–128):** Lazy table initialization in `_ensure_db_ready` and `@app.before_request`.
- **Live Cloud Deployment:** Live production URL on **Render**, Procfile (Gunicorn multi-worker configuration), and Admin Dashboard (/admin) tracking active metrics.

Key Technical Explanations:

- **Referential Integrity:** `ondelete='CASCADE'` guarantees no orphaned links exist when users or teams are deleted.
- **Production Connection Pooling:** `pool_size=10`, `max_overflow=20`, and `pool_pre_ping=True` test connections before queries to prevent cloud timeout drops.
- **Cold-Start Resilience:** Catches sleeping database instances on Render free tier and retries initialization during the first HTTP request instead of crashing the container.

■■ MEMBER 3 SPEAKING SCRIPT (Word-for-Word Walkthrough)

"Thank you [Member 2]. I will explain the **Backend Route Architecture, Relational Database Layer, and Production Deployment Pipeline**.

Under the hood, all incoming requests are orchestrated in `app.py` using Flask's Application Factory pattern. Here is how our data layer and hosting infrastructure operate:

1. Relational Schema & Integrity (models.py): We designed 4 interconnected models: User, Link, Team, and TeamMember. Relationships use `ondelete='CASCADE'` so deleting a user or team cleanly purges dependent records without orphan data.

2. PostgreSQL Engine & Connection Pooling: On our cloud server, we run PostgreSQL with SQLAlchemy 2.0 and the high-performance psycopg (v3) C-binary driver. We configured connection pooling with `pool_pre_ping=True` and `pool_recycle=1800` to prevent stale dropped connections common in cloud hosting.

3. Cold-Start Resilience: In cloud PaaS environments like Render, managed databases may sleep when idle. Rather than allowing the app container to crash on boot, our `_ensure_db_ready` hook lazily verifies table schemas on the first incoming request.

4. Live Cloud Production: LinkVault is deployed live on Render from our GitHub repository using Gunicorn as our WSGI HTTP server with multi-worker pre-fork concurrency (Procfile).

This concludes our technical presentation. We would be glad to answer any questions or perform a live code inspection."

4. Mentor Q&A; Defense Matrix & Setup Checklist

Direct answers for common questions asked by project guides and evaluators:

Mentor Question	Speaker	Recommended Direct Technical Answer
"How does your project prevent SQL Injection?"	Member 2 (Backend)	We use SQLAlchemy ORM which uses parameterized queries across all endpoints. User parameters are never concatenated into raw SQL strings; they are bound as variables by the database driver.
"Why doesn't the search reload the page?"	Member 1 (Frontend)	We filter existing DOM card elements client-side in JavaScript. We use <code>window.history.replaceState</code> to reflect the search query in the browser URL without triggering a GET reload.
"What happens if the cloud database connection drops?"	Member 3 (DevOps/DB)	We configured <code>pool_pre_ping=True</code> and <code>pool_recycle=1800</code> in SQLAlchemy engine options. It tests connection liveness before executing queries and recycles connections before cloud timeouts.
"How do you separate Admin access from regular users?"	Member 2 (Backend)	The User model contains a role field ('admin' 'user'). We use custom view decorators checking <code>current_user.is_admin</code> before serving routes under /admin.
"How is the app deployed and kept running?"	Member 3 (DevOps/DB)	It is hosted on Render PaaS using Gunicorn WSGI server with 2 workers and 4 threads defined in our Procfile, connected via continuous deployment directly to our GitHub repository.
"Can a member access another team's private links?"	Member 2 (Backend)	No. Every team endpoint verifies membership in the <code>team_members</code> join table. If <code>is_member(current_user.id)</code> returns False, the endpoint immediately aborts with HTTP 403 Forbidden.

■ PRE-PRESENTATION 5-MINUTE READINESS CHECKLIST

- **Browser Tab 1 (Main Demo):** Open live Render URL. Start on the public Landing Page in logged-out mode.
- **Browser Tab 2 (Incognito):** Open a private window to demonstrate joining a team workspace via invite code.
- **VS Code Tabs to Open:** `templates/landing.html` (M1), `templates/index.html` (M2), `models.py` & `app.py` L55–125 (M2 & M3), `Procfile` (M3).
- **Target Time:** Member 1 (2 min) + Member 2 (2.5 min) + Member 3 (2.5 min) = ~7 minutes total.